1. Quick Sort

Definition

Quick Sort is a divide and conquer sorting algorithm.

It selects a pivot element, partitions the array into two parts, and recursively sorts them.

Steps

Select a pivot element.

Place smaller elements on the left of pivot.

Place larger elements on the right of pivot.

Recursively apply quick sort on both parts.

Example

Array:

8 3 1 7 0 10 2

Choose pivot = 7

Left: 3 1 0 2

Right: 8 10

Sorted → 0 1 2 3 7 8 10

Algorithm

Copy code

```
QUICKSORT(A, low, high)
1. if low < high
2. pivot = PARTITION(A, low, high)
3. QUICKSORT(A, low, pivot-1)
```

4. QUICKSORT(A, pivot+1, high)

C Code

C

Copy code

```
#include <stdio.h>
int partition(int a[], int low, int high){
   int pivot = a[high];
   int i = low - 1;

   for(int j=low;j<high;j++){
      if(a[j] < pivot){
         i++;
         int temp=a[i];
         a[i]=a[j];
         a[j]=temp;
      }
   }
   int temp=a[i+1];
   a[i+1]=a[high];
   a[high]=temp;
   return i+1;
}
void quickSort(int a[], int low, int high){
   if(low < high){
```

```
        int pi = partition(a,low,high);
        quickSort(a,low,pi-1);
        quickSort(a,pi+1,high);
    }
}
```

Time Complexity

Case

Complexity

Best

O(n log n)

Average

O(n log n)

Worst

$O(n^2)$

Space Complexity

O(log n)

2. Priority Queue

Definition

A Priority Queue is a queue where each element has a priority value.

The element with highest priority is removed first.

Example

Element

Priority

A

3

B

1

C

2

Removal Order → A → C → B

Operations

Insert element

Delete highest priority element

Peek element

Algorithm (Insert)

Copy code

1. Start

2. Insert element according to priority

3. Arrange queue

4. Stop

Applications

CPU scheduling

Dijkstra algorithm

Printer queue

3. Stack and Applications

Definition

A Stack is a linear structure that follows LIFO (Last In First Out).

Example

Stack operations:

Push → Insert element

Pop → Remove element

Diagram

Copy code

Top

| 30 |

| 20 |

| 10 |

Stack Algorithm

Push

Copy code

1. if TOP = MAX-1

2. Stack overflow

3. else

4. TOP = TOP + 1

5. STACK[TOP] = item

Pop

Copy code

1. if TOP = -1
2. Stack underflow
3. else
4. item = STACK[TOP]
5. TOP = TOP -1

Applications

Expression evaluation

Function calls

Undo operations

Parenthesis checking

4. Queue and Applications

Definition

A Queue is a linear data structure that follows FIFO (First In First Out).

Example

Insert → Rear

Delete → Front

Copy code

Front → 10 20 30 ← Rear

Queue Algorithm

Enqueue

Copy code

1. if rear == MAX-1
2. Overflow
3. else
4. rear = rear +1
5. queue[rear] = item

Dequeue

Copy code

1. if front > rear
2. Underflow
3. else
4. item = queue[front]
5. front = front +1

Applications

Printer queue

CPU scheduling

BFS traversal

5. Types of Queue

1. Linear Queue

Simple FIFO queue.

2. Circular Queue

Last position connects to first.

Copy code

1 → 2 → 3

↑ ↓

5 ← 4

Circular Queue Algorithm

Copy code

rear = (rear +1) % size

3. Double Ended Queue (Deque)

Insertion and deletion allowed from both ends.

Types

Input restricted deque

Output restricted deque

6. Binary Search

Definition

Binary Search searches an element in a sorted array by repeatedly dividing the search interval.

Example

Array

2 5 8 12 16

Search = 8

Middle = 8 → Found

Algorithm

Copy code

1. start = 0
2. end = n-1
3. while start <= end
4. mid = (start+end)/2
5. if a[mid] == key return mid
6. if key < a[mid] end = mid-1
7. else start = mid+1

C Code

C

Copy code

```
int binarySearch(int a[], int n, int key){
   int start=0,end=n-1;

   while(start<=end){
      int mid=(start+end)/2;

      if(a[mid]==key)
         return mid;

      else if(key<a[mid])
         end=mid-1;
```

```
        else
            start=mid+1;
    }

    return -1;
}
```

Complexity

Time Complexity → O(log n)

Space Complexity → O(1)

7. Bellman Ford Algorithm

Bellman–Ford algorithm finds shortest path from a source vertex in a graph even with negative weights.

Steps

Initialize distance = ∞

Source distance = 0

Relax all edges V-1 times

Check negative cycle

Time Complexity

O(VE)

8. Floyd Warshall Algorithm

Floyd–Warshall algorithm finds shortest paths between all pairs of vertices.

Formula

Copy code

```
D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

Time Complexity

$O(V^3)$

9. Graph and Types

A Graph consists of vertices and edges.

Copy code

```
A ---- B
|      |
C ---- D
```

Types

Undirected Graph

Directed Graph

Weighted Graph

Cyclic Graph

Acyclic Graph

Applications

Network routing

Social networks

Maps

10. Linear Search

Definition

Linear Search searches element sequentially one by one.

Example

Array

4 7 2 9

Search = 2

Check → 4 → 7 → 2 → Found

Algorithm

Copy code

1. for i=0 to n-1
2. if a[i] == key
3. return i
4. if not found return -1

C Code

C

Copy code

```
int linearSearch(int a[], int n, int key){
   for(int i=0;i<n;i++){
      if(a[i]==key)
      return i;
   }
```

```
    return -1;
}
```

Complexity

Time → O(n)

Space → O(1)